

2. Migración a persistencia en PostgreSQL

- Configura una base de datos PostgreSQL (puedes usar Docker).

- Configuramos el .properties

```
src > main > resources > application.properties
1  spring.mvc.pathmatch.matching-strategy=ant_path_matcher
2
3  spring.datasource.url=jdbc:postgresql://localhost:5432/blueprintsbd
4  spring.datasource.username=admin2026
5  spring.datasource.password=admin2026*
6  spring.datasource.driver-class-name=org.postgresql.Driver
7
8  spring.sql.init.mode=never
```

- Levantamos PostgreSQL con Docker

```
C:\Users\juanj\OneDrive\Desktop\LAB04_ARSW>docker run --name blueprints-postgres -e POSTGRES_DB=blueprintsbd -e POSTGRES_USER=admin2026 -e POSTGRES_PASSWORD=admin2026* -p 5432:5432 -d postgres:16
6e42f5eba8549282419472f733c9123ec40aae1a20729fba2ed7e6553559e25a
C:\Users\juanj\OneDrive\Desktop\LAB04_ARSW>
```

Container CPU usage ⓘ

0.12% / 2200% (22 CPUs available)

Container memory usage ⓘ

29.58MB / 14.98GB

Show charts

Q Search

Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	blueprints-postgres	6e42f5eba854	postgres:16	5432-5432 ↗	0.06%	14 seconds ago	

- Procedemos a crear las tablas de la base de datos

```
PROBLEMS OUTPUT TERMINAL PORTS SONARQUBE
C:\Users\juamp\OneDrive\Desktop\LAB04_ARSN>docker exec -it blueprints-postgres psql -U admin2026 -d blueprintsbd
psql (16.12 (Debian 16.12-1.pgdg13+1))
Type "help" for help.

blueprintsbd=# CREATE TABLE blueprints (
author VARCHAR(100) NOT NULL,
name VARCHAR(100) NOT NULL,
PRIMARY KEY (author, name)
);
CREATE TABLE
blueprintsbd=# CREATE TABLE points (
id SERIAL PRIMARY KEY,
author VARCHAR(100) NOT NULL,
blueprint_name VARCHAR(100) NOT NULL,
x INTEGER NOT NULL,
y INTEGER NOT NULL,
FOREIGN KEY (author, blueprint_name)
REFERENCES blueprints(author, name)
ON DELETE CASCADE
);
CREATE TABLE
blueprintsbd=#
```

```
blueprintsbd=# \dt
          List of relations
Schema | Name   | Type  | Owner
-----+-----+-----+-----
public | blueprints | table | admin2026
public | points  | table | admin2026
(2 rows)

blueprintsbd=#
```

- Agregamos dependencias

```
44
45  ✓ <dependency>
46    <groupId>org.springframework.boot</groupId>
47    <artifactId>spring-boot-starter-jdbc</artifactId>
48  </dependency>
49
50  ✓ <dependency>
51    <groupId>org.postgresql</groupId>
52    <artifactId>postgresql</artifactId>
53  </dependency>
54  </dependencies>
55
```

- Implementa un nuevo repositorio **PostgresBlueprintPersistence** que reemplace la versión en memoria.

```

1 package edu.eci.arsw.blueprints.persistence;
2
3 import edu.eci.arsw.blueprints.model.Blueprint;
4 import edu.eci.arsw.blueprints.model.Point;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.jdbc.core.JdbcTemplate;
7 import org.springframework.stereotype.Repository;
8
9 import java.util.*;
10
11 @Repository
12 public class PostgresBlueprintPersistence implements BlueprintPersistence {
13
14     @Autowired
15     private JdbcTemplate jdbcTemplate;
16
17     @Override
18     public void saveBlueprint(Blueprint bp) {
19
20         String insertBlueprint = "INSERT INTO blueprints(author, name) VALUES (?, ?)";
21
22         jdbcTemplate.update(insertBlueprint,
23             bp.getAuthor(),
24             bp.getName());
25
26         String insertPoint = "INSERT INTO points(author, blueprint_name, x, y) VALUES (?, ?, ?, ?)";
27
28         for (Point p : bp.getPoints()) {
29             jdbcTemplate.update(insertPoint,
30                 bp.getAuthor(),
31                 bp.getName(),
32                 p.x(),
33                 p.y());
34         }
35     }
36
37     @Override
38     public Blueprint getBlueprint(String author, String name)
39         throws BlueprintNotFoundException {
40
41         String checkBlueprint = "SELECT COUNT(*) FROM blueprints WHERE author=? AND name=?";
42         Integer count = jdbcTemplate.queryForObject(
43             checkBlueprint,
44             requiredType: Integer.class,
45             author,
46             name);
47     }
48 }

```

- Mantenemos el contrato con “*BlueprintPersistence*”

Respuesta: Mantenemos el contrato ya que los métodos tienen la misma firma, lanza las mismas excepciones y devuelve los mismos tipos.

- En swagger creamos el blueprint

POST /blueprints

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{  "author": "juan",  "name": "apartamento",  "points": [    {      "x": 2,      "y": 6    }  ]}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  'http://localhost:8080/blueprints' \  -H 'accept: */*' \  -H 'Content-Type: application/json' \  -d '{    "author": "juan",    "name": "apartamento",    "points": [      {        "x": 2,        "y": 6      }    ]  }'
```

Request URL

```
http://localhost:8080/blueprints
```

Server response

Code

Details

201

undocumented

Response headers

```
connection: keep-alivecontent-length: 0date: Fri, 20 Feb 2026 00:43:04 GMTkeep-alive: timeout=60
```

Responses

Code

Description

Links

Hacemos Pruebas curl

```
C:\Users\juanp>curl -s http://localhost:8080/blueprints | jq
[
  {
    "author": "juan",
    "name": "apartamento",
    "points": [
      {
        "x": 2,
        "y": 6
      }
    ]
  }
]

C:\Users\juanp>curl -s http://localhost:8080/blueprints/juan | jq
[
  {
    "author": "juan",
    "name": "apartamento",
    "points": [
      {
        "x": 2,
        "y": 6
      }
    ]
  }
]

C:\Users\juanp>curl -s http://localhost:8080/blueprints/juan/apartamento | jq
{
  "author": "juan",
  "name": "apartamento",
  "points": [
    {
      "x": 2,
      "y": 6
    }
  ]
}

C:\Users\juanp>
```